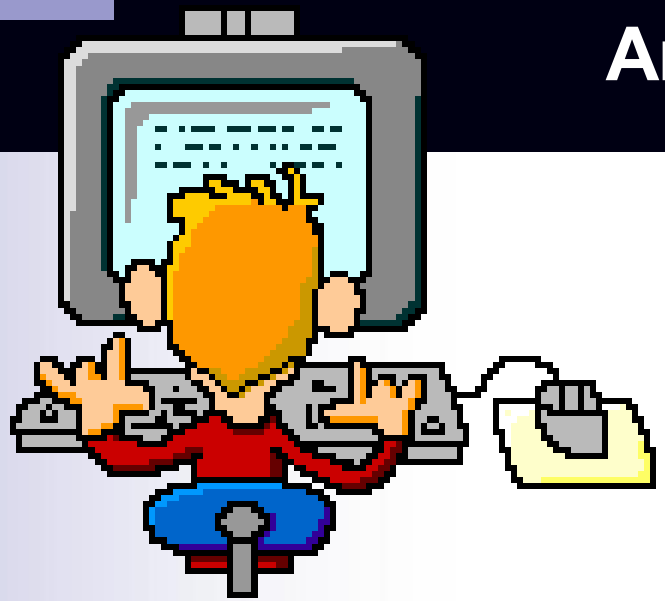
CMP5332
Object-Oriented Programming

Arrays and Collections



Lecture Outlines

- Arrays
- The ‘Arrays’ Helper class
- Collection Types
  - List Collection Types – (ArrayList)
  - Map Collection Types – (HashMap)
- Processing Arrays and Collections
  - Iterating Over Arrays (with and without using the index)
  - Iterating over Collections
    - ArrayList
    - HashMap
- The Collection Helper Class

Arrays

```
String[] days = new String {"Monday", "Tuesday", "Wednesday", "Thursday",
"Friday", "Saturday", "Sunday"};
```

```
String[] days = {"Monday", "Tuesday", "Wednesday", "Thursday", "Friday",
"Saturday", "Sunday"};
```

```
String[] days = new String [7];
days[0] = "Monday";
days[1] = "Tuesday";
days[3] = "Wednesday";
days[4] = "Thursday";
```

```
int[] scores;
char[] tiles;
Person[] persons;
char[][] board;
```

The ‘Arrays’ helper class

To make it more convenient to use arrays, the Java standard library has a “helper class” named `Arrays`. This class is in the package named `java.util`, so we must import it:

```
import java.util.Arrays;
import java.util.*;
```

```
Arrays.sort(days);
Arrays.toString(days)
Arrays.fill(days, "Pizza");
```

Collections

A collection is any data type or data structure that can hold multiple values.

Java has many useful built-in collection types and collection data structures, known together as the **Java collections framework**.

These include:

- The **List type**, and the `ArrayList` and `LinkedList` data structures.
- The **Set type**, and the `HashSet` and `TreeSet` data structures.
- The **Map type**, and the `HashMap` and `TreeMap` data structures.

All of these are in the `java.util` package, so you must import them, either individually or with a wildcard import declaration `import java.util.*;`.

List Collection Type

A **list** is an ordered sequence of values.

In Java, those values must all have the same type; for example:

- `List<String>` means a list of strings, while
- `List<Integer>` means a list of integers.

The syntax for creating a list is like below:

```
List<String> foods = new ArrayList<String>();
```

Instead of `int`, `double`, `char` or `boolean`, you must use a boxed type like `Integer`, `Double`, `Character` or `Boolean`.

List Collection Type

The example below create a list and call some of its methods:

```
List<String> foods = new ArrayList<>();
foods.add("aubergine");
foods.add("broccoli");
foods.add("cabbage");
foods.get(0)
    "aubergine" (String)
foods.get(2)
    "cabbage" (String)
foods.size()
    3 (int)
foods.isEmpty()
    false (boolean)
foods.set(2, "ice cream");
foods.add(0, "ice cream");
foods.remove("broccoli");
foods.toString()
    "[ice cream, aubergine, ice cream]" (String)
```

https://docs.oracle.com/javase/8/docs/api/java/util/List.html

Map Collection Type

A **map** is another word for a dictionary

It contains pairs of "`keys`" and "`values`", so you can use a key to look up its value.

In Java, the keys must all have the same type, and the values must all have the same type;

- for example, `Map<String, Integer>` means a map from strings to integers,
- while `Map<String, Person>` means a map from strings to Person objects.

Map Collection Type

The example below create a map and call some of its methods:

```
Map<String, String> phoneNumbers = new HashMap<>();
phoneNumbers.put("Alice", "555-1234");
phoneNumbers.put("Robert", "555-6789");
phoneNumbers.get("Alice")
    "555-1234" (String)
phoneNumbers.size()
    2 (int)
phoneNumbers.isEmpty()
    false (boolean)
phoneNumbers.containsKey("Alice")
    true (boolean)
phoneNumbers.containsKey("Simon")
    false (boolean)
phoneNumbers.containsValue("555-1234")
    true (boolean)
phoneNumbers.put("Alice", "555-4321");
phoneNumbers.toString()
    "{Robert=555-6789, Alice=555-4321}" (String)
```

Processing Arrays and Collections

Iterating Over Arrays

```
String[] days = new String[] { "Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday" };
```

Using index to iterate over the array. In this case, we need a variable (usually named `i`) to keep track of the current index:

```
for(int i = 0; i < days.length; i++) {
    String day = days[i];
    System.out.println("Day #" + i + " is " + day);
}
```

Iterating Over Arrays

```
String[] days = new String[] { "Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday" };
```

Most of the time you use an array, you want to do something for each value in the array. There is a simple syntax for this in Java:

```
for(String day : days) {
    System.out.println(day + " is a day of the week.");
}
```

Iterating Over Collections - ArrayList

```
List<String> foods = new ArrayList<>();
foods.add("aubergine");
foods.add("broccoli");
foods.add("cabbage");
```

The same `for` loop syntax can be used to iterate over a list or a set:

```
for(String food : foods) {
    System.out.println(food + " is on the menu.");
}
```

You can also do something *for each index* in a list – e.g., print the list

```
for(int i = 0; i < foods.size(); i++) {
    String food = foods.get(i);
    System.out.println("Food #" + i + " is " + food);
}
```

The ‘Collections’ helper class

```
public List<String> getFoods() {
    return Collections.unmodifiableList(foods);
}
```

The `unmodifiableList` method returns a “view” of the list which prevents other classes from modifying it. Similarly, the `unmodifiableSet` and `unmodifiableMap` methods protect sets and maps from modification by other classes.

Iterating Over Collections - Map

```
import java.util.Map;

public class MapPrinter {
    public static void printKeys(Map<String, String> map) {
        [redacted]
    }
}
```

```
Map<String, String> phoneNumbers = new HashMap<>();
phoneNumbers.put("Alice", "555-1234");
phoneNumbers.put("Robert", "555-6789");
```

`Printkeys(phoneNumbers)`

[“Alice”, “Robert”]

Scrabble

```
char[] tiles = {'A','B','C','D','E','F','G','H','I','J','K','L','M','N',
 'O','P','Q','R','S','T','U','V','W','X','Y','Z'};
```

```
scores = new int[] {1,3,3,2,1,4,2,4,1,8,5,1,3,1,1,3,10,1,1,1,1,4,4,8,4,10};
```

```
Map<Character, Integer> tileScores = new HashMap<>() {{
    put('A', 1);
    put('B', 3);
    put('C', 3);
    put('D', 2);
}};
```

```
for (int i=0; i<tiles.length; i++) {
    tileScores.put(tiles[i], scores[i]);
}
```

Scrabble

```
Map<Character, Integer> tileScores = new HashMap<>() {{
    put('A', 1);
    put('B', 3);
    put('C', 3);
    put('D', 2);
}};
```

```
public int scoreForTile(char tile) {

    tileScores.get(tile);
}
```

Scrabble

```
public int scoreForTile(char tile) {

   tileScores.get(tile);
}


public int scoreForWord(String word) {
  char[] tiles = word.toCharArray();

              ..scoreForTile(tile);
}


public String highestScoringWord(List<String> words) {
    ..scoreForWord(word);
}
```